D3 · Dataset: remove duplicates, split safely

Find exact and near copies with MinHash and LSH, keep one per family, then split with no leaks.

COMP 329 course staff

Rice University

Spring 2027 · Assignment D3

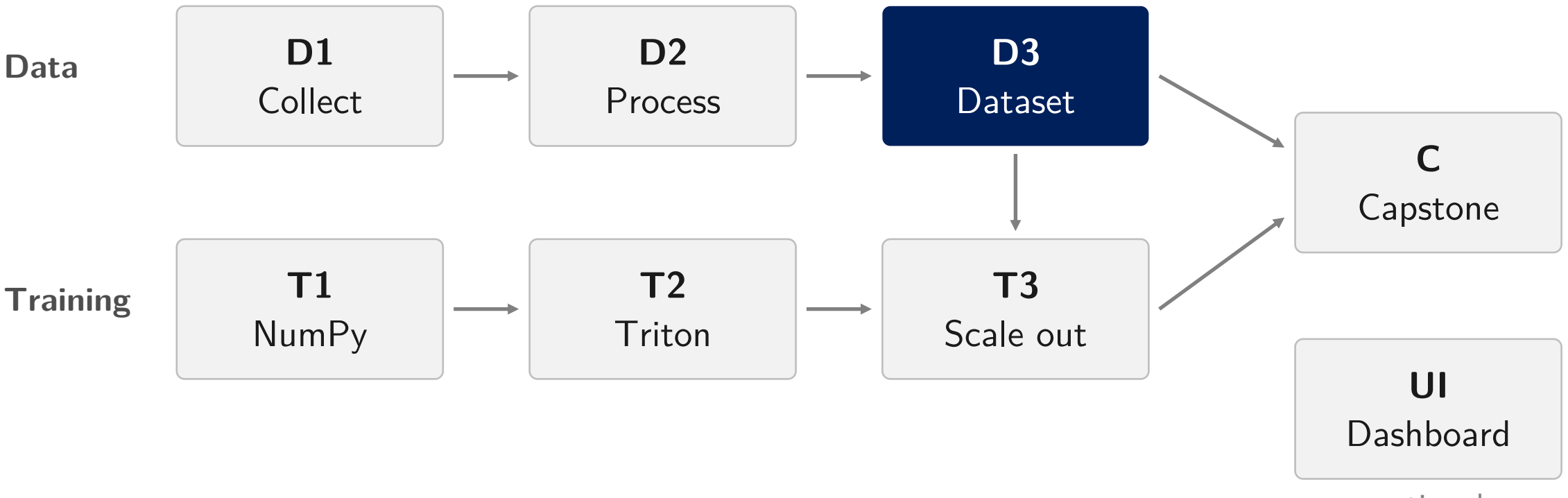
Release week 4

Checkpoint week 6

Final week 7

10-12 hours

Where this assignment fits



In our pipeline

This assignment: `runs/inputs/processed` → **D3** → `datasets/byte`, `datasets/bpe`

What you will build, and why

Make sure the model never trains on a copy of an article it is later tested on.

1 **Leaks inflate scores**

A near copy in training makes validation look better than it is.

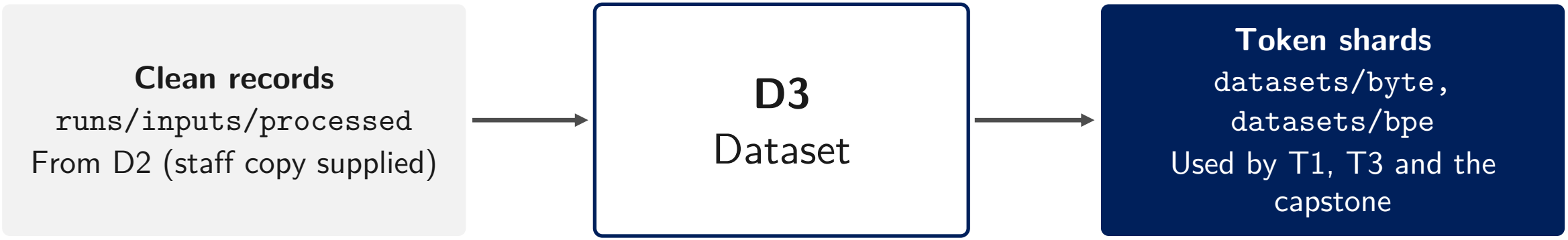
2 **Scale without all-pairs**

LSH proposes likely pairs instead of comparing every pair.

3 **Order matters**

Deduplicate first, then split; the reverse leaks copies.

What goes in, what comes out



Files you produce

`deduplication.stats.json`
Exact and near removals, confirmed edges

`datasets/*/manifest.json`
Shards, splits, tokenizer checksum

`datasets/*/*.bin`
uint16 token shards, BOS … EOS per document

From text to copy families

1

Shingles

Width-5 word sequences: "one two", "two three" ...

→

2

MinHash

Keep the minimum hashed value per seeded function.

→

3

LSH bands

Hash each band; same bucket = candidate pair.

→

4

Confirm

Exact Jaccard $\geq$ 0.82 turns a candidate into an edge.

→

5

Components

Keep the smallest ID of each connected family.

Key idea

A bucket collision is only a candidate. If A~B and B~C, keep one of A, B, C even if A and C are not similar.

A deterministic signature

```
p = 2**61 - 1
x = blake2b(shingle, 8) % p   # little-endian
salt = sha256(f'{seed}:{i}')
a = le(salt[:8])   % (p-1) + 1
b = le(salt[8:16]) % p
sig[i] = min((a*x + b) % p)

band key = f'{j}:' + sha256('12,7,93')
```

1 **Python integers**

uint64 multiplication can overflow; Python ints don't.

2 **Prefix the band index**

Stops collisions between different band positions.

3 **Split after dedup**

Order survivors by SHA256(seed:`doc_id`); 10% validation, 10% test.

4 **Tiny-corpus audit**

Check all pairs once to measure recall and leaks; it's not the main algorithm.

The staff input after deduplication

32 → 24

documents

4 + 4

exact + near removals

0

cross-split leaks

Key idea

Recall on the audit is 1. Both byte and BPE contracts pass, and the BPE tokenizer is fitted on training text only.

What you write, what you get

You write

`minhash_signature`

`starter/dedup.py`

`band_keys`

`starter/dedup.py`

`deduplicate`

`starter/dedup.py` · 6 numbered TODOs

Provided

- shingles and Jaccard similarity
- Spark session and plan helpers
- BPE tokenizer, shard writer and checker
- mmap loader and public tests

Ground rules

Keep the signatures · don't edit `starter/pipeline_lab` · run through `course.py` or `lab.py`.

Build it in this order

1 — **Signature**

Reproduce the deterministic MinHash.

2 — **Bands**

Contiguous bands → indexed bucket keys.

```
check d3 --checkpoint
```

3 — **Families**

Exact hashes, candidates, Jaccard, components.

4 — **Split + Spark**

Leak-free splits; candidates confirmed in Spark.

```
check d3 --systems
```

5 — **Shards**

Build and inspect byte and BPE datasets.

```
lab.py d3
```

Check

Done when: All 4 tests pass with Spark; 32 → 24 with zero leaks.

If something goes wrong

Same text, different signature
Check: Seed and byte order, lowercase shingles, overflow.

One pair appears many times
Check: Deduplicate candidates shared across bands.

A drop points to a dropped ID
Check: Resolve through the final component root.

Validation has a training copy
Check: Dedup before splitting; inspect missed pairs.

Offsets are twice expected
Check: Manifest offsets count tokens, not bytes.

"10" vs "2"
Check: Compare IDs as strings: "10" sorts first.

The performance study

How do more bands trade work for recall?

Predict first

More bands raise collision probability and candidate work. Predict both.

64 permutations: 8 / 16 / 32 bands
same input, threshold, seed

BPE merges
changes the vocabulary, not a pure speedup

How to measure: Report candidates vs. all pairs, recall, precision, drops, leaks, build time and output size.

How you are graded

20	40	16	24
Checkpoint	Final correctness	Performance	Analysis

Correctness: hash/LSH 10 · copy resolution 10 · splits/contracts 10 · individual walkthrough 10

Check

Done when: All 4 tests pass with Spark; 32 → 24 with zero leaks.

Your first ten minutes

```
$ bash setup.sh
$ source .venv/bin/activate
$ python course.py preflight
$ python course.py inputs --output runs/inputs
$ python course.py check d3 --checkpoint
$ python lab.py d3 --input runs/inputs/processed \
    --output runs/d3-first --merges 32
```

Your folder

`03_d3_dataset/`

Handout:
`03_d3_dataset/README.md`